

Key takeaways

Version control system

- Version Control System (VCS) là hệ thống quản lý các phiên bản.
- Có 3 loại VCS:
 - **Local:** lưu ở máy cá nhân.
 - **Centralize:** lưu ở một máy chủ tập trung.
 - **Distributed:** lưu ở nhiều máy khác nhau.

Git

Git? Tại sao dùng Git

- **Git** được viết bởi **Linus Torvalds**, cha đẻ của Linux
- **git** là từ viết **sai chính tả** (có chủ đích) của get, do get đã được dùng rồi
- Dùng git do **nhu cầu quản lý phiên bản và làm việc giữa nhiều người với nhau**

Git và GitHub

Git	GitHub
Là một phần mềm	Là một dịch vụ web
Cài trên máy của bạn	Host trên website
Là một commandline tool	Là công cụ có giao diện
Là công cụ quản lý phiên bản, đưa file vào Git repository	Là nơi để upload Git repository lên

Có các tính năng của Version Control System	Có các tính năng của Version Control System và một số tính năng khác
---	--

Ba vùng trạng thái

Sau khi khởi tạo, git sẽ có 3 vùng trạng thái:

- Working directory
- Staging
- Repository

Các câu lệnh thường dùng

Câu lệnh	Công dụng
git init	Khởi tạo một Thư mục Git mới (Working Directory), chứa các file tạo mới chưa được commit (untracked files).
git config user.name "<name>" git config user.email "<email>"	Cấu hình cho 1 repo
git config --global user.name "<name>" git config --global user.email "<email>"	Cấu hình cho toàn bộ máy tính (default)
git add <file_name>	Thêm 1 file vào vùng staging
git add .	Thêm toàn bộ file vào vùng staging
git status	Xem trạng thái file: - File màu xanh: vùng staging - File màu đỏ: vùng working directory
git commit -m"message"	Chuyển tất cả các file đang ở trạng thái sẵn sàng commit (staging), sang trạng thái commit (tạo một phiên bản - Repository)
git log	Kiểm tra lịch sử commit

Javascript basic

Đuôi file và cách chạy file

- Đuôi file: **file_name.js**
- Chạy lệnh bằng lệnh: **node <path_file_name>**

Biến

Biến = biến thiên = thay đổi.

Định nghĩa

Biến dùng để lưu trữ giá trị các dữ liệu, hay các đối tượng. Giá trị của biến **có thể được thay đổi, cập nhật** trong quá trình ứng dụng hoạt động. Mục này trình bày các vấn đề về biến như tên biến, khai báo và khởi tạo biến ...

Quy tắc đặt tên biến

Tên biến do bạn đặt và khai báo, tên biến cần đảm bảo quy tắc sau:

- Tên biến được tạo ra bởi các ký tự chữ, số, **_** và **\$**.
- Tên **không được phép bắt đầu bằng số** (chỉ bắt đầu bằng ký tự chữ hoặc \$ hoặc _).
- **Không được** chứa các ký hiệu đặc biệt như ký hiệu toán học, logic (như +, -, *, >, < ...).
- **Không được** chứa khoảng trắng.
- **Không được** đặt tên biến trùng với các từ khóa dành riêng cho ngôn ngữ Javascript liệt kê ở dưới:

abstract	else	instanceof	super
boolean	enum	int	switch
break	export	interface	synchronized
byte	extends	let	this
case	false	long	throw
catch	final	native	throws
char	finally	new	transient
class	float	null	true
const	for	package	try
continue	function	private	typeof
debugger	goto	protected	var
default	if	public	void
delete	implements	return	volatile
do	import	short	while

double in static with

Khai báo và khởi tạo biến

Để khai báo biến, ta có 2 từ khoá: var và let.

Ta có thể khai báo và gán giá trị ngay. Ví dụ:

```
var name = "Playwright Viet Nam";  
let major = "Học automation test từ chưa biết gì";
```

Hoặc khai báo và gán giá trị sau:

```
var name;  
let major;  
name = "Playwright Việt Nam";  
major = "Học automation test từ chưa biết gì";
```

Lưu ý: tên biến không được trùng nhau.

Sử dụng biến

Ta có thể sử dụng biến ngay trong hàm console.log:

```
console.log(name);
```

Thay đổi giá trị

Để thay đổi giá trị của biến, ta thực hiện gán lại giá trị của biến sang giá trị khác mà không cần từ khoá var/let.

```
var name = "Playwright";  
name = "Playwright Viet Nam";
```

Hằng số

Hằng = hằng số = không thay đổi.

Định nghĩa

Hằng số là các giá trị không thay đổi được sau khi khai báo.

Quy tắc đặt tên hằng

Giống với quy tắc đặt tên biến. Lưu ý tránh từ khoá.

Khai báo và khởi tạo hằng

Hằng số cần gán giá trị ngay tại thời điểm khai báo:

```
const framework = "Playwright";
```

Thay đổi giá trị

Hằng số không thể thay đổi được giá trị. Nếu cố tình thay đổi, sẽ gây ra lỗi:

```
const course = "Youtube";
```

```
course = "Udemy";
```

```
//      ^
```

```
// TypeError: Assignment to constant variable.
```

Lời khuyên khai báo

- Đối với các giá trị không thay đổi, dùng `const`.
- Khai báo biến sử dụng `let` để dễ kiểm soát phạm vi truy cập. Không dùng `var`.

Các nhóm

Javascript chia kiểu dữ liệu ra thành 2 nhóm:

- **Nhóm kiểu dữ liệu nguyên thủy**: đây là các kiểu dữ liệu được xây dựng sẵn trong Javascript. Có 8 kiểu dữ liệu trong nhóm này. Đó là:
 - `boolean`
 - `null`
 - `undefined`
 - `number`
 - `string`
 - `symbol`
- **Nhóm kiểu dữ liệu đối tượng**: đây là các kiểu dữ liệu cho người dùng tự tạo ra.

Nhóm kiểu dữ liệu nguyên thủy (primitive)

Kiểu logic - boolean

Biểu diễn giá trị đúng/sai:

- Giá trị đúng: `true`
- Giá trị sai: `false`

Thường dùng trong các biểu thức của câu điều kiện.

Ví dụ:

```
var isPlaywright = true;  
let isCypress = false;  
const isSelenium = false;
```

Kiểu null - rỗng

Kiểu null đại diện cho giá trị rỗng - không chứa gì cả. Ví dụ:

```
var lazy = null;
```

Kiểu undefined

Kiểu undefined đại diện cho giá trị chưa được định nghĩa. Ví dụ:

```
var name = undefined;  
console.log(name);
```

```
let age;  
console.log(age);
```

Kiểu number

Kiểu number đại diện cho các giá trị kiểu số. Trong Javascript, không phân biệt kiểu dữ liệu số nguyên, số thực mà gọi chung tất cả là “kiểu number”

```
var num1 = 12;  
let num2 = 15.893;  
const num3 = 0;
```

Kiểu string

Kiểu string đại diện cho các giá trị kiểu chuỗi, dạng văn bản. Có thể lưu giá trị kiểu chuỗi nằm trong cặp ngoặc kép hoặc ngoặc đơn:

```
var str1 = 'simple';  
let str2 = "double";  
const str3 = 'triple';
```

Các kí tự đặc biệt

Đối với các kí tự đặc biệt, bạn cần dùng kí tự escape (\) đứng trước

- Kí tự \: \\
- Kí tự nháy đơn: \'
- Kí tự nháy kép: \"
- Kí tự xuống dòng: \n
- Kí tự đầu dòng: \r
- Kí tự tab: \t

Toán tử so sánh (comparison operator)

Comparison operator = toán tử so sánh.

Toán tử so sánh dùng để so sánh giá trị của 2 vế. Toán tử so sánh sẽ trả về true nếu đúng, trả về false nếu sai.

Toán tử	Ví dụ	Kết quả	Ý nghĩa
>	2 > 5	false	Phép so sánh lớn hơn
<	2 < 5	true	Phép so sánh nhỏ hơn
>=	2 >= 5	false	Phép so sánh lớn hơn hoặc bằng
<=	2 <= 5	true	Phép so sánh nhỏ hơn hoặc bằng
==	5 == '5' 2 == 2	true true	Phép so sánh bằng, chỉ so sánh giá trị, không so sánh về kiểu dữ liệu
===	5 === '5' '5' === '5'	false true	Phép so sánh bằng, so sánh cả về giá trị và về kiểu dữ liệu
!=	5 != '5' 2 != 3	false true	Phép so sánh khác, chỉ so sánh về giá trị, không so sánh về kiểu dữ liệu
!==	5 !== '5' '5' !== '5'	true false	Phép so sánh khác, so sánh cả về giá trị và về kiểu dữ liệu

Unary operator

1. **i++** (Hậu tố tăng/giảm):

- **Ý nghĩa:** Giá trị của biến **i** được sử dụng trước, sau đó mới tăng/giảm giá trị.
- **Cách hoạt động:**
 - Sử dụng giá trị hiện tại của **i**.

- Sau đó, tăng/giảm giá trị của `i` lên 1.

```
let i = 5;

console.log(i++); // In ra 5, nhưng i sau đó tăng lên 6

console.log(i);   // In ra 6
```

1. `++i` (Tiền tố tăng/giảm):

- **Ý nghĩa:** Giá trị của biến `i` được tăng/giảm trước, sau đó mới sử dụng giá trị đã thay đổi.
- **Cách hoạt động:**
 - Tăng/giảm giá trị của `i` lên 1.
 - Sau đó, sử dụng giá trị mới của `i`.

```
let i = 5;

console.log(++i); // Tăng lên 6, sau đó in ra 6

console.log(i);   // In ra 6
```

Arithmetic operator

- Dùng tính toán giá trị biểu thức
- Các phép toán: `+`, `-`, `*`, `/`

Conditional

Khái niệm

Câu điều kiện kiểm tra nếu điều kiện đúng thì sẽ thực thi code trong khối lệnh, giúp thực hiện các logic rẽ nhánh phức tạp.

Có một số loại câu điều kiện thường dùng:

- `if`
- `if...else`
- `if...else...if`
- `Switch...case`

Điều kiện `if`

Cú pháp điều kiện `if`:

```
if (condition) {  
    // code block  
}
```

Trong đó, `condition = true` thì sẽ chạy đoạn `code block`.

Loops

- Dùng để thực hiện một đoạn logic một số lần nhất định
- Cú pháp: `for(<khởi tạo>; <điều kiện dừng>; <điều kiện tăng>) { // code }`
- Ví dụ

```
for (let i = 1; i <= 5; i++) {  
    console.log("Giá trị của i là: ", i);  
}  
// 1  
// 2  
// 3  
// 4  
// 5
```

Format code

- Mac: Option + Shift + F
- Window: Alt + Shift + F

Chú ý: setting VSCode path về đường dẫn mặc định